PROJECT SUMMARY

Budget Buddy 2.0

An AI-assisted personal-finance web application for students

Le Anh Thu | Independent Computer Science Project | September 2026

76 *respondents surveyed* **15/15** *automated tests passing* **2** *languages* **2** *currencies*

Budget Buddy began as the final project for Harvard's CS50 Python course: a command-line program that calculated a balance and ranked income and expense items for one person, on one machine, with no way to see a month at a glance.

Version 2.0 rebuilds that idea as a full web application. Every user owns an isolated account and a relational-database record of their own; budgets, alerts, and savings goals are visual rather than textual; and an AI assistant reads a user's own spending history and explains it in plain language - in whichever language and currency that user has chosen.

The project applies material from CS50 Python and CS50 SQL to a real, deployed product, grounded in a 76-person survey of how young people in Vietnam manage their personal finances and where they encounter difficulty.

51.3% of respondents said they track their spending "in their head" and nothing else. Budget Buddy 2.0 is built for exactly that gap - full research on the next page.

WHERE TO LOOK

Video walkthrough: Available in this portfolio

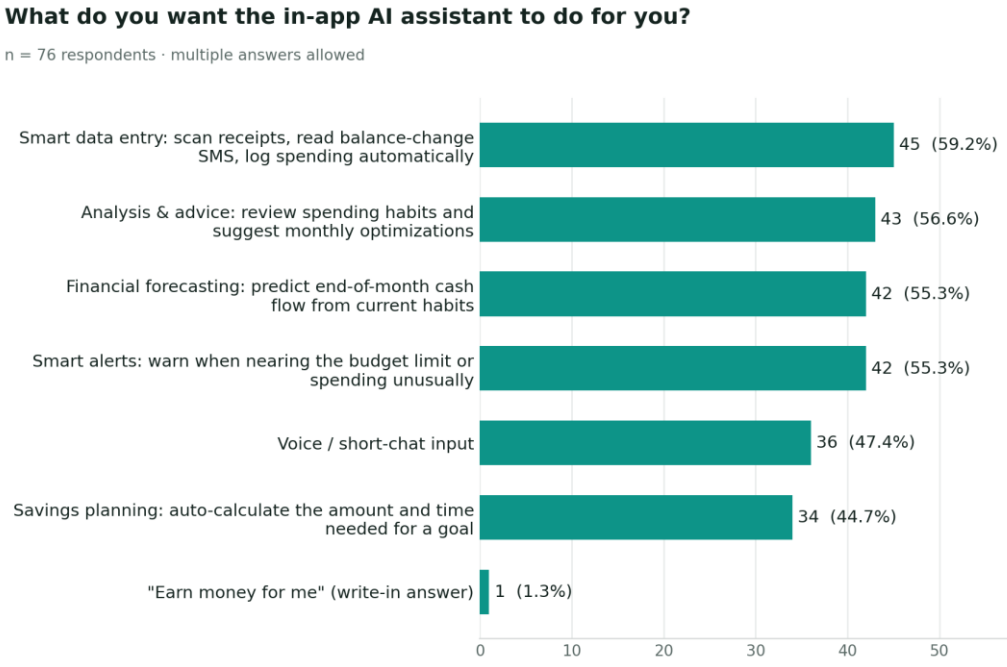
Live demo: https://budget-buddy.io.vn

GitHub repository: github.com/thula1010/budget_buddy_web

From User Research to Product Decisions

To inform the development of version 2.0, I surveyed 76 students and young working people in Vietnam about how they manage money today, what stops them, and what they would want an AI assistant to do. Several questions allowed more than one answer, so figures don't sum to 100%. The findings below directly shaped which features were built first.

What the survey found	What I built because of it
51.3% track money “in their head” only, and 38.2% have never recorded their finances in any form	**A deliberately minimal first interaction — recording a transaction has to beat doing nothing**
44.7% frequently forget to record small everyday expenses	**API-enabled receipt scanning and message reading to reduce manual data entry**
40.8% don't know how to allocate their spending sensibly	**AI Coach that reads a user's own history and explains it, plus category budgets**
55.3% want smart alerts for over-budget or unusual spending	**A budget-overrun alert that fires once, at the moment a limit is first crossed**
56.6% want offline operation and bank / e-wallet sync	**Offline-first support prioritized for Phase 1; bank/e-wallet synchronization planned for Phase 2.**

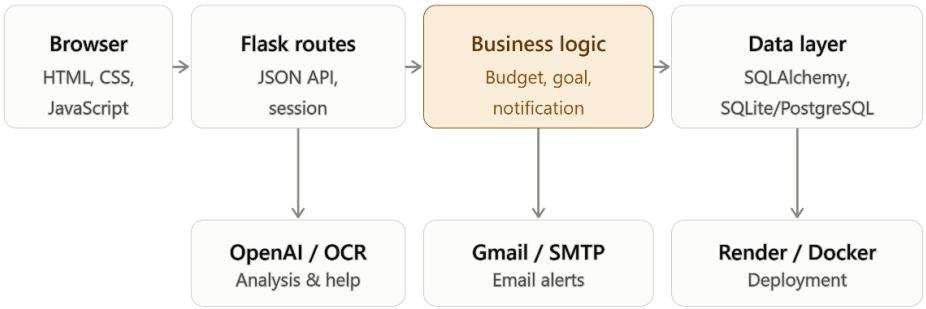


What respondents wanted the in-app AI assistant to do (n = 76, multiple answers allowed).

Method: an online questionnaire distributed to students and young working people, concentrated in Hanoi (44%) and Ho Chi Minh City (16%), with the remainder spread across other provinces.

Technical Design & What I Built

A layered monolithic web app: Flask handles sessions and the JSON API, SQLAlchemy maps the business models to SQLite locally and PostgreSQL in deployment, and the browser handles rendering and the language–currency layer. AI/OCR and email are optional integrations - the core app still works when they aren't configured.



Logical architecture:

browser → Flask routes → business logic → data layer, with AI/OCR and email as optional side integrations.

1. Full-stack architecture

Flask and SQLAlchemy on SQLite locally, PostgreSQL in deployment - the same models run in both environments. JavaScript in the browser owns rendering, data entry, and the language/currency adaptation layer, with Chart.js for spending visualizations.

2. An end-to-end bilingual, multi-currency system

Language and currency are independent user choices that apply everywhere at once - the interface, the API, the AI Coach, and every email - not just static labels. VND stays the internal base unit, so switching currency never rewrites a stored transaction.

3. First-breach alert logic

A budget-overrun email fires only at the moment spending crosses the limit - not on every transaction afterward. A separate emailAlertSent field ensures that the interface reports success only when the email provider accepts the send request.

CORE FEATURE SET

- Isolated per-user accounts & data
- Income & expense categorization
- Monthly budgets with first-breach alerts
- Savings goals with deadlines & tracking
- Spending analytics & visualizations
- AI Coach for personalized spending analysis

Results, Evidence & Reflection

The test suite passes 15 of 15 cases, covering authorization, transactions, goals, OCR, data upgrades, and the localization/currency contract described on the previous page.

Test group	Observed result	What it establishes
Threshold overrun	Mail is triggered only when state changes from within-limit to over-limit	Repeated alerts are prevented for later expenses
Localization	EN–VI content and VND–USD amounts follow the selected preferences	Email content and currency formatting match the selected user preferences.
UI status	The “sent” modal doesn't open when no mail provider is configured	No false success is ever reported
Data compatibility	User upgrades preserve existing rows and relationships	Safe to deploy onto a database that already holds data

Building Budget Buddy 2.0 taught me that a feature list means nothing without evidence. The requirements were informed by user research and technical design decisions, and the tests check written contracts—not just the happy path.

KNOWN LIMITATIONS

- The exchange rate is a static configuration value, not a live market rate.
- End-to-end email delivery hasn't yet been tested against a real mailbox.
- Receipt scanning and message reading depend on external APIs being configured.

WHERE TO LOOK

Video walkthrough: Available in this portfolio

Live demo: https://budget-buddy.io.vn

GitHub repository: github.com/thula1010/budget_buddy_web